

FallGuard: Individual Work Contribution and Reflection

Yaorong Huang

Group 7, COMP6131 Internet of Things Essentials

Faculty of Applied Sciences, Macao Polytechnic University, Macao SAR, China

Abstract—I served as group leader and the principal implementation and integration contributor to FallGuard. My work connected quality-aware temporal modeling, experiment tooling, portable model export, independent Raspberry Pi inference, persistent cloud delivery, complete operator workspaces, and the final technical report and presentation. This document records the tasks, implementation problems, solutions, and concrete deliverables from that work.

I. YAORONG HUANG

A. Role and overall responsibility

I served as group leader and principal implementation and integration contributor. I carried out most of the project’s code implementation and took primary responsibility for the technical manuscript and web presentation. My work connected the research model to a running IoT system: model design and preprocessing, experiment tooling, portable inference, Raspberry Pi deployment, local persistence, cloud synchronization, operator workspaces, and reproducibility materials. I also coordinated the interfaces between the team’s data, training, cloud, and reporting tasks and completed the final integration and consistency checks.

B. Model design, implementation, and experiment tooling

I implemented MaskedBiMamba, combining temporal attention, independent forward and backward Mamba processing, training-time frame masking, and quality-weighted pooling. I defined how normalized skeleton coordinates and joint confidence enter the temporal model and how the separate quality features determine frame weights. This required implementing the input representation, branch alignment, feature fusion, pooling, and classification interface, then connecting those components to the training and evaluation code. The resulting model provides a common input/output contract for research evaluation and deployment.

I implemented and organized the model-comparison and component-ablation tooling, including configuration files, random seeds, checkpoint selection, metric calculation, and result aggregation. The completed evaluation contains 60 primary runs across five models, four folds and three seeds; 120 ablation runs across ten variants; and nine runs in the separate TCNTE reproduction track. My work covered the complete comparison pipeline, from recorded configurations and checkpoint selection to aggregated results and figures. I also organized primary robustness evaluation, external Le2i evaluation, and replay of the deployed confirmation rules.

I connected experiment outputs to the manuscript’s tables and plots, checking the distinction between window predictions, video predictions, merged events, and confirmed alerts. The evaluated model achieves the highest primary window precision at the common threshold (0.6048). Under 30% frame removal it retains window F1 of 0.6276, or 98.1% of its clean score, compared with 0.4361 for TCNTE. These results gave the model-development work a clear evaluation outcome and a concrete rationale for its deployment.

C. Model export and independent Raspberry Pi inference

I converted the selected temporal checkpoint into a portable two-input ONNX model and maintained its input shapes, normalization, class order, model card, and checksums. I implemented the portable inference path so that the Pi classifier does not depend on the CUDA training extensions. The deployment accepts a 30-step skeleton sequence and a separate three-feature quality sequence, matching the model used in the experiment pipeline.

I integrated USB-camera and video acquisition, YOLOv8s-pose extraction accelerated by Hailo, a CPU pose fallback, and local ONNX classification. I implemented timestamped one-second windows advanced every 0.2 seconds, interpolation, pose-quality checks, and handling of missing or non-monotonic observations. I also implemented the local alert transitions and connected them to stored records. This work turned the trained classifier into a service that can acquire input, classify windows, maintain state, and preserve results on the Raspberry Pi without an external network.

I supported the paired hardware evaluation and incorporated its measurements into the deployment and report. The complete local pipeline reached 20.56 frames/s with Hailo versus 1.73 frames/s with CPU pose extraction under the same YOLOv8s-pose comparison. Temporal classification remained on the Pi CPU. The deployed result therefore reflects acceleration of the complete implemented pipeline, together with a usable CPU fallback.

D. Persistent delivery and complete operator workspaces

I integrated SQLite record storage, an independent out-box, and cloud synchronization. The design preserves record identifiers and original capture times, commits cloud records before acknowledgement, and safely retries pending records after reconnection. I connected these components to the cloud archive and the private device-status bridge. The recorded

delivery tests matched all 85 evaluated records; a 30-second upload interruption retained 17 records, with the queue first observed empty 2.83 seconds after recovery.

I completed the final three-page local and cloud workspaces: detection overview, history, and device diagnostics. Beyond a basic prediction display, I implemented live preview, result freshness, historical probability charts, filters, pagination, CSV export, queue status, and resource diagnostics. I added explicit handling for incomplete poses, expired results, paused cameras, and lost device connections, so that the current display follows the operating state while saved history remains available.

I verified service startup and shutdown, archive queries, authentication, delivery, and recovery using the software suite and the actual page scripts. Deployment updates preserved existing SQLite data, checked service health and source hashes, and retained rollback copies. I also restored the browser-camera demonstration alongside the offline workspace, including its landmark-to-classifier adapter and current-model inference. This provided both the deployed Pi workflow and a computer-based demonstration entry.

E. Problems encountered and solutions implemented

One integration problem was maintaining the same model input in training and deployment. I made the pose normalization and separate quality input explicit, aligned the timestamped buffer with the training window definition, and checked preprocessing and PyTorch/ONNX numerical agreement. The two-input export comparison showed a maximum absolute logit difference of approximately 3.6×10^{-7} .

A second problem was keeping inference and recorded results available during network interruption. I separated camera inference, local storage, the outbox, and synchronization, then exercised restart, retry, and lost-acknowledgement behavior. UUID-based deduplication and acknowledgement after durable storage preserved the recorded history during recovery.

A third problem was that the earlier interface did not provide enough information for independent operation and troubleshooting. I replaced it with the full local/cloud workspaces and added freshness checks, history inspection, and diagnostics. I also corrected the preprocessing order in the missing-joint test path and updated the result selection, figures, and manuscript together. Resolving these issues required changes across model code, deployment services, interfaces, and documentation.

F. Manuscript, presentation, and reproducibility publication

I took primary responsibility for writing and revising the IEEE LaTeX report: method notation and equations, architecture diagrams, experiment protocols, results, system implementation, figures, captions, and references. I revised the argument around the model's precision, frame-loss tolerance, edge throughput, and recoverable cloud delivery, while keeping the numerical statements consistent with the saved results. I also maintained the Chinese mathematical and system-workflow guides and the project brief, providing explanations for both technical review and presentation preparation.

I developed the twelve-slide GitHub Pages presentation, its interactive comparison charts, the searchable archive of 1,853 aggregate result rows, bilingual speaker notes, projection controls, presenter console, and rehearsal timer. I synchronized the data, explanations, source files, and published outputs across the report, guides, presentation, deployed software, and repository.

I organized the public reproducibility release: experiment and deployment source, dependency files, generic configuration and service templates, selected weights, ONNX model, asset hashes, dataset provenance, and run instructions. I used the 30-test software suite, numerical export checks, release manifests, and GitHub verification workflow to check the handover. I prepared the final report source, independent contribution document, presentation PDF, and fixed repository submission version as a coherent set of deliverables.

G. Deliverables and reflection

Main deliverables: `code/src/fallbench/`, `code/configs/`, `code/scripts/`, `deployment/`, `artifacts/models/`, `webui/`, `tests/`, `results/`, `manuscripts/final-report/`, `documentation/zh/`, `docs/`, and the `reproducibility/submission` guides.

My contribution extended from algorithm implementation to the final running system and its submitted evidence. The most demanding part was keeping these layers consistent: the model input used by training, the exported classifier, the device buffer, the cloud record, and the value shown to an operator. Working through that chain strengthened my ability to debug across components and to connect a research result to a maintainable IoT implementation. I also learned to treat experiment organization, recovery behavior, technical writing, and reproducibility as substantive engineering work, with concrete outputs that can be inspected and run.